

Research Note: LLD Practice Platform

CipherSchools Engineering Assignment — Shubham Dubey

1. The Learner Problem

Low-Level Design is a skill where reading about it and actually doing it are wildly different. A learner can watch a dozen Strategy pattern videos, read through a Parking Lot walkthrough, and still freeze when asked to design one from scratch. The gap isn't knowledge — it's practice and feedback.

- No fast feedback loop — unless a senior engineer is sitting next to you, nobody tells you if your class responsibilities are off or your abstractions are leaky.
- Multiple valid designs exist for the same problem, making self-evaluation frustrating since there's no single answer to compare against.
- Resources are mostly passive — you read or watch, but don't actually attempt, submit, and get told where you went wrong.
- Evidence of attempts disappears — no history to look back at and see if you're improving over time.

2. Existing Approaches & Tools

HelloInterview — Content platform by FAANG engineers with a structured delivery framework for LLD and guided walkthroughs. Good content, but fundamentally a reading resource — you follow along, you don't submit your own design and get feedback.

AlgoMaster.io — Closest to what this assignment envisions. Timed AI-led mock interviews with four stages (requirements, entities, class design, code), scored out of 10 per stage with written feedback and attempt history. But scoring is a single number per stage — not broken down by design dimensions like coupling or extensibility — and feedback doesn't quote your own submission back at you.

LLDCanvas — Visual LLD practice with an OOP-aware UML diagram editor, 110+ problems, and timed interview mode. Leans heavily into diagram tooling — the "lots of UI, weak practice logic" pattern the helping guide warns against. Feedback model is not well documented.

LLDMastery — Interactive playground with 43 problems, a 4-step framework, and AI review. Heavy investment in visual diagram builder and in-browser code execution over the practice loop itself. AI feedback gives general observations rather than structured rubric-based evaluation.

CodeZym — Machine-coding approach: write Java code, submit against test cases, LeetCode-style. Solid testing flow, but evaluation is "does it pass the tests" — no feedback on design quality. A solution with terrible coupling but passing tests gets a perfect score.

LLDProblems — AI-assisted practice where AI generates class skeletons from your visual sketches. 100+ problems with smart hints. The approach helps you write code rather than evaluate your design thinking. Feedback focuses on hints, not structured design-quality assessment.

3. Key Gaps

- Feedback is either a single opaque score or vague comments. Nobody breaks it down by design dimensions with evidence quoted from your submission.
- Platforms over-invest in tooling (diagram editors, compilers, collaboration) and under-invest in the practice loop itself.
- Code-first platforms evaluate whether code works, not whether design is good. Passing tests doesn't mean your class structure is sound.
- No structured retry loop showing improvement. AlgoMaster saves history, but you can't compare two attempts to see what changed.
- The evidence-to-feedback link is missing everywhere — feedback says "coupling is high" but doesn't point to the specific classes causing it.
- Deterministic vs AI evaluation isn't thoughtfully separated. Nobody splits cheap deterministic checks from expensive AI judgement.

4. Product Direction

The product direction targets one core idea: structured, evidence-linked feedback per design dimension, with a retry loop that shows improvement.

- Plain text as submission format — the smallest format that still gives enough evidence of design quality, avoiding the diagram-tooling trap.
- Fixed rubric with multiple dimensions (responsibilities, coupling/cohesion, encapsulation, abstraction, extensibility, edge cases, explanation). Each dimension: score + evidence from submission + concern + suggestion.
- Split deterministic checks (non-empty, minimum length, class/interface tokens, duplicate hash) from AI judgement (design quality assessment). Don't waste an AI call on an empty submission.
- Store every attempt and show a delta view — what changed between attempts, did scores per dimension go up or down. Makes improvement visible.
- Intentionally boring architecture — monolith, synchronous evaluation, state machine (Submitted/Evaluating/Completed/Failed), no queue or Redis. The judgement to not over-engineer is itself part of what's being evaluated.

North star: *a learner submits a design, gets told exactly where and why it's weak (with evidence from their own submission), and comes back to try again — seeing measurable improvement each time.* None of the tools I found do this well today.

5. Scope & What's Intentionally Left Out

To keep the MVP focused within the 2-day window, several things are deliberately excluded — not because they're unimportant, but because they don't improve the practice loop and would eat time that should go into the domain model and evaluation rubric.

- No authentication system. A single demo user is hardcoded. Auth is a solved problem that adds no signal to an LLD assignment.
- No diagram editor. The decision to use plain text is a feature, not a shortcut — it keeps the focus on design thinking rather than drawing. A diagram format can be added later without changing the domain model (Change Test A).
- No job queue or Redis. Evaluation runs synchronously with a state machine. If this scaled, the evaluator would be the first component to extract — but that is a design-note discussion, not something to build now.
- No reference-solution matching. The rubric evaluates design quality, not correctness against a single right answer. Two structurally different designs can both score well if their responsibilities, abstractions, and trade-offs are sound.